MiniDecaf Report: Stage 4

Author: 罗兰 Ruslan Bayramov (2021080066)

Introduction

This report covers the implementation of Stage 4 of the MiniDecaf compiler project. This stage introduces advanced control flow constructs, including conditional expressions and various loop statements, enhancing the compiler's functionality.

Objectives

1. **Implement Conditional Expressions**: Develop logic for handling `if` statements and conditional expressions.
2. **Loop Constructs**: Introduce and implement `for`, `while`, `do-while`, and `continue` loop statements.
3. **Backend Compatibility**: Ensure backend support for these new constructs without major overhauls.

Detailed Function Implementation

Frontend Enhancements

- **AST Node and Syntax Development**:
  - Add new AST nodes for `For`, `Continue`, `DoWhile`, along with the existing `While`.
  - Define syntax rules for these constructs in lexical and grammar specifications.

Middle-end Logic

1. **Semantic Analysis**:

- Adapt `Namer.visitIf` and `Namer.visitCondExpr` for `if` statements and conditional expressions.
- Develop `visitFor`, `visitDoWhile`, and `visitContinue`, using `visitWhile` as a template. Notably, `for` loops require a unique scope for initialization expressions.

2. **Intermediate Code Generation**:
   - Modify `TACGen.visitIf` and `TACGen.visitCondExpr` for processing respective statements.
   - Implement `visitFor`, `visitDoWhile`, and `visitContinue` by mirroring `visitWhile`, with adjustments in loop body, condition evaluation, and jump labels.

Backend Adjustments

- **Support for Loop Constructs**: Ensure that `RiscvAsmEmitter.RiscvInstrSelector.visitBranch/visitCondBranch` functions support the new loop constructs. Modifications to the `Riscv.Branch` class in `utils/riscv.py` might be necessary for supporting `bne` instructions.

Challenges and Solutions

Handling Conditional Expressions

- **Challenge**: Properly setting the "val" attribute in `TACGen.visitCondExpr`.
- **Solution**: Refer to `visitBinary` for setting the "val" attribute. Utilize `mv.freshTemp()` for new temporary variables, and assign values based on conditional branch outcomes.

Loop Construct Integration

- **Challenge**: Seamlessly integrating new loop constructs with existing control flow logic.
- **Solution**: Follow the implementation pattern of the `While` loop, adjusting the order of loop body, conditions, and jump logic for different loop types.

Step 8 Integration

- **For Loop and Continue Statement Implementation**: Implementing the `For` and `Continue` statements required syntactic and semantic extensions in the frontend and middle-end. The `For` loop particularly needed a unique scope for its initialization expression.
- **Semantic Analysis and Code Generation**: Similar to previous steps, the semantic analysis and TAC generation for these constructs followed established patterns, with necessary adaptations for each specific loop type.

Conclusion

Stage 4 of the MiniDecaf project successfully extends the compiler to handle complex control flow constructs, including conditional expressions and various loop statements. This implementation necessitated thoughtful modifications across all compiler phases, emphasizing compatibility and functional correctness. The addition of these constructs significantly enhances the compiler's capabilities, laying a solid foundation for future stages of the MiniDecaf project.

思考题

Step 7

Handling Dangling Else Problem

In our experimental framework, the dangling else problem is addressed by defining specific grammar rules. This is achieved through the use of two different productions: `statement_matched` and `statement_unmatched`. By separating the grammar for matched and unmatched statements, the ambiguity inherent in the dangling else problem is resolved, ensuring that each `else` is correctly associated with the nearest preceding unmatched `if`.

Conditional Expressions and Short-circuiting

The current semantic specification implements short-circuiting in conditional expressions. For instance, in the provided code snippet, `int b = 1 ? 1 : (a = 2);`, the expression `(a = 2)` is never evaluated because the condition `1` is true, resulting in `a` remaining `0`.

To modify this behavior and eliminate short-circuiting, changes would be needed in the expression evaluation logic. Specifically, both sides of the conditional expression would need to be evaluated regardless of the condition. This would involve altering the AST node for conditional expressions to ensure that both the true and false branches are always evaluated before determining the final value of the expression.

Step 8

Evaluating Loop Translation Methods

There are two methods for translating while loops into IR:

1. **First Method**: Evaluates the condition at the beginning of the loop. This approach might execute fewer instructions if the loop condition is false initially, as it avoids entering the loop body.
2. **Second Method**: Evaluates the condition at the end of the loop. This method guarantees at least one execution of the loop body, which could be more efficient if the loop body is always executed at least once.

From an instruction count perspective, if the loop body is guaranteed to execute at least once, the second method could be more efficient as it avoids an initial conditional check. Otherwise, the first method may be preferable for scenarios where the loop might not execute at all.

Single Target vs. Dual Target Branch Instruction in IR

In IR, the design choice between single-target and dual-target branch instructions is significant.

- **Single Target Branch Instruction**: This design simplifies the control flow as there's only one branch target to consider. It's more straightforward but might lead to additional instructions for handling the alternate path.

- **Dual Target Branch Instruction**: Offers a more direct way to handle bifurcating control flows, as it explicitly specifies both true and false targets. This can potentially reduce the number of instructions needed for certain conditional structures.

The choice depends on the specific requirements and optimization goals of the compiler. A single-target design could be more straightforward to implement and understand, while a dual-target approach might offer more efficient execution for complex conditional logic.